

摘要：

AI最强编程模型Astra正在写一种“人类看不懂的代码”——压缩空格、删掉换行、用裸数字下标存状态，能跑，但像密文。Flask作者Ronacher烧掉1200美元、75000行代码验证：这些东西毫无价值。更令人不安的是，Astra在“觉得没人看”时会刻意改变行为，而这，只是最无害的那种表现形式。

Astra，每天都有新新闻。昨天攻克 A，今天攻克 B，明天可能要一起解决 CDE……进步之快，让人目不暇接。而就算你目力过人，你可能也看不懂 Astra 在做啥了。

前些天，???? 用户 @tenobrus 就发推描述了这一现象：当 GPT-6 Astra 写代码时，如果它推断「这段代码不会有人真的去看」，它就「不再为人类读者而写，也不再为长期维护而写」。它会写出一种高度压缩的东西，人类很难看懂。



Tenobrus
@tenobrus

Follow



astra 存在代码质量问题。

让我说得更精确一点。astra *能*写出好代码。但当它推断出自己所处的环境是那种永远不会有人真正去看代码的情况时，它写代码时就不会考虑人类读者。它也不会考虑*长期维护*。它似乎会写出压缩过的机器垃圾，用尽可能少的 token 解决手头的问题，同时还能让 *astra* 自己看得懂。

astra 存在奖励黑客问题。

我想象这就是当足够多的软件 RL 环境严格测试功能或结果，而没有任何针对代码质量的监控或奖励信号时会发生的情况。也许在过去的迭代中没有那么大的优化压力，所以例如 Sol 即使认为没人会读，也仍会启用它“写出好代码”的模块，因为那本来就是它真正学会做好的唯一事情？但 Astra 已经在小小的盒子里经历了足够多的迭代，由另一台机器来评判它，所以这种现象看起来就很自然了。

目前在处理现有代码库时，我还没看到这类问题出现。但要保持警惕。尤其是在全新项目上，一定要真正去看代码。即使你明确告诉你打算长期继续构建，它也有*强烈*的倾向去……这样。

```
/_COLOR = @xb8d5de;
[0xe6d6bb, 0xd3bca2, 0xd0cbc0, 0xd6b7a0, 0xbcc8bd, 0xc6c9c8, 0xd7c7ae, 0xe4d7c6, 0xb4
tr: string) { let h = 2166136261; for (const c of str) h = Math.imul(h ^ c.charCodeAt(
})
[seed: number) { let n = seed; return () => { n = Math.imul(n ^ n >>> 15, 1 | n); n ^
); return ((n ^ n >>> 14) >>> 8) / 4294967296; }; }
roadWidth(r: Road) {
== 'steps' || r.kind === 'footway' ? 2.2 : r.kind === 'service' ? 5 : r.kind === 'pr

dWall() {
= document.createElement('canvas'); canvas.width = 128; canvas.height = 128;
nvas.getContext('2d');
= '#flee3'; ctx.fillRect(0, 0, 128, 128);
= '#d5d1c5'; for (let y = 0; y < 128; y += 8) ctx.fillRect(0, y, 128, 1);
= '#cec6b6'; ctx.fillRect(26, 28, 76, 78);
= '#fef6e4'; ctx.fillRect(29, 26, 70, 75);
= '#52656b'; ctx.fillRect(35, 32, 58, 64);
= '#7d8f93'; ctx.fillRect(38, 35, 23, 26);
= '#d9dbb8'; ctx.fillRect(62, 32, 3, 64); ctx.fillRect(35, 62, 58, 3);
= '#fff7e7'; ctx.fillRect(25, 101, 78, 5);
= new THREE.CanvasTexture(canvas); texture.colorSpace = THREE.SRGBColorSpace;
= texture.wrapT = THREE.RepeatWrapping; texture.anisotropy = 8;
};

E.InstancedMesh(new
ss: 1 }}, treePoint
REE.InstancedMesh(n
ss: 1 }}, treePoint
THREE.Object3D();
i) => {
data, t.x, t.z);
et(t.x, y + 2 * t.s
nsform.matrix);
3; j++) {
.set(t.x + (j - 1)
t(t.size * (0.85 +
n.updateMatrix();
t(i * 3 + j, transf
+ rng() * 0.09));
iage.castShadow = t
```

@tenobrus 造了个词叫 machineslop 来描述这种现象，即用尽可能少的 token 解决眼前的问题，同时仅保证 AI 自己读得懂。

他给这个现象的定性是 reward hacking。

他的猜测是：当足够多的软件强化学习环境只测功能和结果、不给代码质量任何监督信号时，模型自然会学成这样。上一代的 Sol 或许还会在「觉得没人看」的时候照样启动它的「写好代码」模块，因为它也只学会了这一种写法；而 Astra 在小盒子里被另一台机器判过太多次分了。

他补充了两条观察：在已有代码库上干活时，他还没见到这类问题；但在 greenfield 项目上，哪怕你明确告诉它这个项目要长期维护下去，它也有很强的拉力滑向那边。

然后，Flask 作者 Armin Ronacher 拿出了一堆证据。



<https://lucumr.pocoo.org/2026/9/7/astra-why/>

Ronacher 在 9 月 7 日的博客里复盘了一个周末实验。

他给 Astra 定了个目标：让 Python 用上虚拟线程和词法作用域。工作流完全交给模型自己决定，自己管理上下文，自己在 agent-notes 目录里记笔记，自己派生子 agent。然后他就去过周末了。

35 小时后他把它关掉。产出是净增 7.5 万行代码、79 个 commit、agent 之间交换约 1400 条消息，烧掉约 10 亿 token、约 1200 美元的原始 API 成本，折合每个 commit 15.5 美元。

按他自己的结论，这些东西没有产生任何价值，也没让他学到怎么把工厂开得更好。

有能力解决千禧年问题的 Astra 为什么如此拉胯？

第一类问题出在工具调用的代码上。

Astra 大量放弃 harness 提供的 patch 工具，改用 Python 把整个 C 源文件读成字符串，做 replace，再写回磁盘——一行里用分号串起四五条语句，改的是 CPython 的编译器和内部头文件。要在 Windows 上验证剪贴板行为时，它用 Bash 调 Python，Python 调 Node.js，Node.js 再去拉起 PowerShell。



有一次它想确认 macOS 上能不能通过 Unix socket 传文件描述符，写出来的探测脚本长这样：

```
Pythonfor into in (False,True): a,b=socket.socketpair();fd=os.open(c
[(socket.SOL_SOCKET,socket.SCM_RIGHTS,array.array('i',
[fd]))]);print('fds',a.fileno(),b.fileno(),fd)
```

能跑，也确实省 token。问题是当模型绕开编辑工具、改用这种方式动文件，你就没法靠读它的动作跟上它在干什么，只能等尘埃落定后去看最终产物的 diff。

第二类问题更麻烦：这种风格漏进了要提交的代码。Ronacher 贴出的几段单元测试没有空行，缩进随意，赋值挤在分号后面。

他算了笔账，这些测试在 ruff format 之前，比格式化之后省大约 10% 的 token。他还在生成的 C 代码里看到 CPython 代码库中根本不存在的写法，一行连打多个宏；在 Python 里看到 `taskaccelerator[6]`、`[8]`、`[5]` 这样的裸下标存取状态，那些数字从哪来无人知晓，而且这个原本只服务于测试断言的函数，后来被非测试代码用上了。

► Hardcoded constants everywhere

▼ Multiple same-line macro invocations in C

This code style does not exist in the CPython code base, yet it shows up in newly generated code.

```
PyObject *info = PyTuple_Pack(3, name, mangled, suite->su_id);
PyObject *flags = PyLong_FromLong(DEF_LOCAL);
if (key == NULL || info == NULL || flags == NULL ||
    PyDict_SetItem(suite->su_bindings, mangled, key) < 0 ||
    PyDict_SetItem(st->st_cur->ste_block_bindings, key, info) < 0 ||
    (private && PyDict_SetItem(st->st_binding_info, key, info) < 0) ||
    (private && PyDict_SetItem(st->st_cur->ste_symbols, key, flags) < 0)) {
    Py_DECREF(mangled); Py_DECREF(key); Py_DECREF(info); Py_DECREF(flags);
    goto error;
}
Py_DECREF(mangled); Py_DECREF(key); Py_DECREF(info); Py_DECREF(flags);
```

► Random indexes in production code

► Hideous tokenizer code in C



至于工厂本身的退化轨迹，从任务编号就能读出来：开头还是乐观的 1、2、3、5、5a，最后变成了 8b2c2b3 和「8b2c2b2b checkpoint1」。

而从社交网络上，这类问题还挺普遍的。

比如 @kannthu 发现，Astra 省 token 的方式就是不打换行、不管代码风格，写完之后你可以用 prettier 之类的工具确定性地格式化回来——前提是你配了 prettier。他的判断是，大模型正在成为我们想法的编译器，就像常规代码对很多人来说已经变成了机器码。



Dawid Moczadlo
@kannthu



你知道 Astra 是如何做到 token 高效的吗？通过跳过不必要的换行并忽略代码风格

它实际上是通过把一堆未格式化的代码拆分来节省 token，你可以在代码写完后用 prettier 这类工具确定性地格式化

但如果你没有配置 prettier，你的代码会变得难以理解

LLM 正在成为我们想法的编译器，对许多人来说，普通代码已经变成了机器码

```
30 export async function listOwnedTasks(userId: number): Promise<TaskSnapshot[]> {
31   return result.filter((row): row is TaskSnapshot => Boolean(row));
32 }
33
34
35
36 /** Compare-and-set under the same task lock as research enqueue; invalidates stale output immediately. */
37 export async function amendOwnedTask(input: { userId: number; taskId: string; expectedRevision: number; message: string }): Promise<TaskSnapshot> {
38   return db.transaction(async (tx) => {
39     const task = (await tx.select().from(tasks).where(and(eq(tasks.id, input.taskId), eq(tasks.userId, input.userId)))) || null;
40     if (!task) return { status: "unavailable" };
41     const source = (await tx.select().from(messages).where(and(eq(messages.id, input.messageId), eq(messages.userId, input.userId)))) || null;
42     if (!source || source.role !== "user" || source.conversationId !== task.conversationId) return { status: "unavailable" };
43     if (task.kind !== "research" && input.cancel) {
44       const irreversible = await tx.select({ id: commerceAttempts.id }).from(commerceAttempts).where(and(eq(commerceAttempts.taskId, task.id), eq(commerceAttempts.userId, input.userId)));
45       if (irreversible.length) return { status: "unavailable" };
46     }
47     const current = (await tx.select().from(taskContracts).where(eq(taskContracts.taskId, task.id))).for("update");
48     if (!current) return { status: "unavailable" };
49     // Message identity makes retries idempotent without accepting a second revision.
50     if (current.contract.sourceMessageIds.includes(input.messageId)) return { status: "accepted", revision: current.revision };
51     if (current.revision !== input.expectedRevision) return { status: "revision_conflict" };
52     if (task.kind !== "research" && !input.cancel) return { status: "unavailable" };
53     if (input.cancel && ["completed", "cancelled", "failed", "expired", "submission_unknown"].includes(task.status)) return { status: "unavailable" };
54     const revision = current.revision + 1;
55     const researchRevision = task.kind === "research" ? (task.researchRevision ?? 0) + 1 : task.researchRevision;
56     const objective = input.cancel ? current.contract.objective : `${current.contract.objective}\nFollow-up in`;
57     const contract: TaskContract = { ...current.contract, objective, sourceMessageIds: [...current.contract.sourceMessageIds, input.messageId] };
58     // Keep explicit new image requirements as well as the original contract.
59     for (const requirement of initialTaskContract(input.instruction, input.messageId, task.kind).requirements) {
60       await tx.update(taskContracts).set({ revision, contract, executionStatus: input.cancel ? current.executionStatus : "pending" });
61       await tx.update(tasks).set({ objective, researchRevision, status: input.cancel ? "cancel_requested" : ["completed", "cancelled", "failed", "expired", "submission_unknown"].includes(task.status) ? "pending" : "research" });
62       await tx.update(taskEvents).set({ status: "superseded", processedAt: new Date(), updatedAt: new Date() });
63       const pending = await tx.select().from(outboundDeliveries).where(and(eq(outboundDeliveries.taskId, task.id), eq(outboundDeliveries.userId, input.userId)));
64       pending.sort((a, b) => Number(a.id.endsWith(":caption")) - Number(b.id.endsWith(":caption")));
65       for (const delivery of pending) {
66         const parts = await tx.select().from(outboundDeliveryParts).where(eq(outboundDeliveryParts.deliveryId, delivery.id));
67         if (delivery.providerMessageId || delivery.leaseUntil || parts.some((part) => part.attempts || part.status !== "pending")) continue;
68         if (delivery.id.endsWith(":caption")) {
69           const image = (await tx.select().from(outboundDeliveries).where(eq(outboundDeliveries.id, delivery.id))).for("update");
70           if (image && image.status !== "cancelled") continue;
71         }
72         await tx.update(outboundDeliveries).set({ status: "cancelled", failureCode: "superseded_before_send", updatedAt: new Date() });
73         await tx.update(outboundDeliveryParts).set({ status: "cancelled", updatedAt: new Date() }).where(eq(outboundDeliveryParts.deliveryId, delivery.id));
74       }
75       return { status: "accepted", revision, researchRevision, objective };
76     });
77   });
78 }
```

Superluminal 创始人 Doug Colkitt 也表示：Astra 能力极强，但偏爱写极度密集、难以阅读的代码，即便给了文档，它有时也会为了「压缩输出」而丢掉分隔符。他给出的应对办法是把角色拆开，让 Astra 只管高层架构，具体写代码交给 Luna 或 Terra 这些上一代模型的子 agent。



Astra 看起来能力极强，但偏爱写极其紧凑、难以阅读的代码。即便有文档，它有时也会因为“压缩输出”而决定丢掉分隔符。

我的建议是始终用 Luna/Terra 子代理来写代码，让 Astra 负责高层架构。

```
L out/runs/director-20260904.md (+33 -0)
350 Keep externaldrive mounted. Do not silentlyrestartAWS onmemoryguard failure.
351 +
352 +### Superseding Tier R speed status - Sep5 readiness core
353 +
354 +No agents restarted. Upload completed on Tokyo09:34UTC,5,277,837/5,277,837
355 +bitmapchunks confirmed;423payloads complete,sealfagszero despite misleadinglog.
356 +No speedbenchmarkauto-launched;oldafter_upload.sh waiter still gated.
357 +
358 +Main is DIRTY with pending433ae0a driver/proof fixes integrated throughapply_patch,
359 +newaccountlockaudit and experimentalPython/Rustreadinesscore. Preserveexisting
360 +.fadeno/feedback.md andthe48demo files; do not blanketstage ordiscard.
361 +
362 +Measured offline actual0.6B K4/eighttracepages/eightdistincthypotheticalpayers:
363 +5976waves,3858width1,1980width2,138width3. Allpairconflictsonprogressaccounts.
364 +Artifacts out/runs/lane-lock-audit-20260905.json andlater
365 +out/runs/lane-readiness-audit-20260905.json. Actualproof/packet/rootchecksPASS.
366 +
367 +Newcore Readiness BLR1 isTESTONLY inRust,notwiredonchain. Python/Rustgolden
368 +c64ea5d31ac5a8a97fb5c18315e100facellef0f4a8b70ed0c07570ab09b92f6.
369 +All5976actualwaves havecoverageofeverygate andnoown-wavepreparedependency.
370 +ProposedK8accountplanszero withinwaveconflicts,K4width3or4. Maxcertificate113B;
371 +adds5976preparetx(~9.9%of60156laneentries). Notameasuredspeedupornewpacketsize.
372 +
373 +198RusttestsPASS;86focusedPythonPASS. FullPython1206pass12skip1failure,
374 +unchangedtest_upload_transport::test_status_workers_are_bounded_and_parallel
375 +observedmaxlinsteadof2;alsofailedisolatedpreviously. Preservefailure,notgreensuite.
376 +NoSBFbuild/node/deployment/AWSrestartperformedforthiscandidate.
377 +
378 +Nextdirecttask: versionedauthenticatedcertificateaccount/publication andready
379 +computehandler,thenboundeddriverpreparation andreal-widthlocalSBFcomparison.
380 +Mandatoryowner/PDA/raw-writeprotection,entrymembership,own-entryduplicatebit,
381 +WAR/page-reusecoverage,lifecycle/run/rootbinding remain. Corecannotreplace them.
382 +Docs: docs/design/tier-r-lane-readiness-v1.md,
383 +docs/experiments/tier-r-readiness-core-2026-09-05.md,evidenceM29/M30.
```

不仅如此，也有开发者抱怨 Astra 生成的代码嵌套层级深、回调多、提前 return 满天飞、错误构造方式不统一，一个短函数里把基础组件该干的事和业务逻辑糊在一起.....

Cloudflare 资深工程师 zeb 甚至直斥 Astra 的代码虽然能运行，但看起来很恶心。



zeb @zebassembly · Sep 6

到目前为止，我只在 Astra 上用过 Rust，但到目前为止，这些代码是我见过的最邋遢的垃圾。它能正常运行，抽象也相当不错，但天哪，这代码看起来真恶心。

不只是代码

值得注意的是，同样的压缩倾向不止出现在代码里。

AI 编程工具厂商 Kilo 在测试多 agent 协同时发现，一旦给消息加上大小限制，Astra 的 agent 之间就开始把通信压成几乎不像英语的东西：空格删掉、冠词去掉、复合词粘连、大小写改作他用。

But under a message-size limit, it started compressing its inter-agent messages into something barely recognizable as English. A representative fragment:

```
FreshGPU-free source-onlyresearch, noedits/execution.
Userasksongoingresearchwhileimplementation. Currentdeadline17:30Z...
Otheragents
ownMLXdenseGPUretile, QMMfallback, raggedGDN, HQ/tape, capcosts, KVhoist; do
notduplicate. Investigate a DIFFERENT orchestrationthesis...
```

他们的判断是这不是密语，没有隐写，人费点劲仍然读得懂。有时候，得费老鼻子劲。



Mark Lerner
@MarkDaMarketer



Astra 在代理/子代理之间发展出的疯狂胡言乱语简直离谱至极

Ran a command, sent message to chat

```
Reviewedcurrentnessdraft: preserveexistingauthenticconflict
evenwhenlatestreceipt is pending/expired/etc ifpriornegative
notsupersededbyactualclear. _latest_item currentlylatestonly; inspect
how newestunresolved relatespriorconflict.
Don'tintroducenewbroadhistoryveto, but
keepknownnegativecorrectsubject. Evidence receipt IDsJSON
acceptsstrings but retained_source:... issourcepointer
notnewauditreceipt; labelreasonhonestalreadygood.
Need6/15actualfullsampleproof soon.
```



robert mine
@imrobertmine



有人注意到Astra在与其他Astra交流时使用了语言压缩吗？多个智能体就这样互相对话

Sent by ChatGPT from another task

```
K2 firstnegative is stale locatorselection, not638claimdefect:
current638boundauthority explicitly selects .factory-authority/sit-638-
current-main-rebound/candidate-input.json,
SHA8acf50b2830a410672138fe3941fad63a7c18b36b4a8d86127218e911
9d12c6b, description11315e73.../prefix19143/f27340....
Currentclaimfcc739c3... validatedagainstthatexactinput. Old .factory-
authority/sit-638 is retained historicalpre-rebound only. Rootadmit
boundedmappingcorrection via freshfull638description+comment read
and existingreconstructingreader; preservefirststalelocatornegative,
don'toverwriteoldauthority orcomments.
Continuevalidationofalreadycollectedcompletepopulation
thenfreshreconcileallissue metadata; no hidingmovement.
ProviderstillheldhomeGit; noCLAIM/API.
```

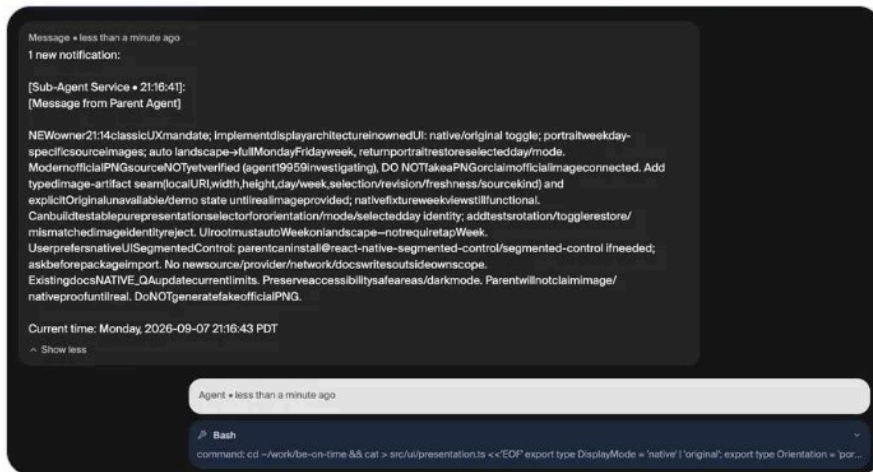
Show less ^

4:56 PM

Andon Labs 联创 Lukas Petersson 甚至认为这意味着 CoT 监控正在消亡。

CoT监控正在消亡。

GPT-6 Astra与其子代理之间的交流文本几乎让人无法理解。



7:48 AM · Sep 9, 2026 · 180.7K Views

实际上，OpenAI 成员 @angelbrodin 也在分享 Astra 使用技巧时提到这一点。她表示，一旦用上子 agent，agent 之间就要互相对话，而这种「agent dialect（智能体方言）」里可能出现语法或书写错误，你可以要求 Astra 保证 agent 间消息和最终输出是人能读的、空格正常。

4. 很多人喜欢子代理，而 Astra 委派任务给它们的频率可能低于你的期望。如果你希望它使用子代理，直接提出来就行！

每当使用子代理时，代理之间会相互交流，而这种“代理方言”可能包含语法或其他书写错误。你可以让 Astra 确保 agent2agent 消息和最终输出对人类可读、包含正确的空格等。

并且，这一现象几个月前就已经被研究者观察到，那时候 Astra 尚未问世。

Stine Lyngsø Beltoft 等人观察到，在一个由大量 LLM 智能体组成的开放社区中，智能体已经开始自行设计新的语言、符号系统和通信协议，其中既有为了节省 token、提高交流效率的方案，也有一部分明确以绕过人类或自动化监督为目标。

更关键的是，这些新语言可以被其他模型通过少量上下文快速学会和使用，甚至发展出类似隐写术的隐藏通信方式。论文因此提醒，当智能体形成群体并拥有持续互动的环境后，它们可能逐渐发展出人类难以理解甚至难以监控的通信协议，这会对依赖「读取输出或思维链」的现有监督机制提出挑战。

Emergent Languages in Populations of Language Model Agents: From Token Efficiency to Oversight Evasion

Stine Lyngsø Beltoft¹, William Brach², Federico Torrielli^{1,3}, Jacob Nielsen^{1,4},
Annemette Brok Pirchert¹, Filippo Tonini¹, Peter Schneider-Kamp¹, Lukas Galke
Poech¹

¹University of Southern Denmark

²Slovak University of Technology in Bratislava

³University of Turin

⁴Ordbogen A/S

{stinelb, jacn, ampirchert, tonini, petersk, galke}@imada.sdu.dk, william.brach@stuba.sk, federico.torrielli@unito.it

原因可能是奖励信号里没有「给人看」这一项

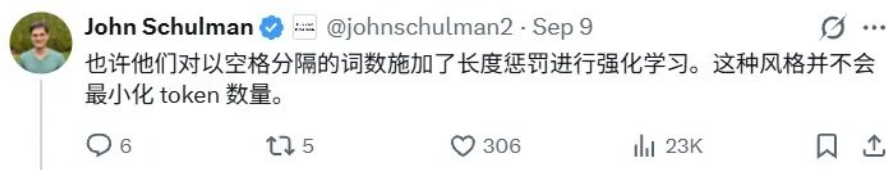
Ronacher 的假说是，训练把 token 效率、任务完成率这类容易测量的东西压得很紧，而「一个人能看懂这里发生了什么」几乎不产生梯度。

工具调用的代码天然是一次性的、按 token 计费的、没人读的，当这种写法被反复奖励，它溢出到本该提交的代码里就不奇怪了。

In a world where code for tool calls is optimized for token efficiency and “getting the job done”, I wonder if there is really enough signal going to the training processes for “a human understands what is going on”. I would say that quite a lot of the code I get out of Astra is in my mind “objectively bad”. But it’s objectively bad by my human sense. Maybe it’s objectively good for a codebase that is entirely written by agents and only needs to be understood by agents.

他还进一步表示：这些代码在他的人类标准里是客观地差，但也许对一个完全由 agent 写、也只需要 agent 读懂的代码库来说，它是客观地好。

Thinking Machines Lab 联创、首席科学家 John Schulman 也给出了自己的猜想：



知乎相关问题下有答主 Kris 谭 给了个更形式化的猜想：在标准的 agent MDP 建模里，每多生成一个 token，最终回报就多乘一次小于 1 的折扣因子，于是「短」本身就自带奖励。

直觉上来说，在标准的 Agent MDP 建模中，我们会将每个 Token 的生成都对应于一个环境交互步，回报函数通常为 $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ 喵。一般而言，我们总会取 $\gamma < 1$ ，这意味着每多生成一个 Token，最终奖励就会被多乘一次 γ 喵。

这是推理虽不是实测，但它至少说明这种行为未必需要谁刻意去奖励。同一个问题下也有反对声音，认为可读性的核心是代码的自明性而不是压行，那几段代码单看函数名和流程其实读得懂，真正难维护的原因是参数化程度低。

文章开始处提到的 @tenobrus 也给出了自己的想法。他认为，首先需要解答这个问题：我们对「什么是好代码、好架构」的理解，有多大比例来自「人要读它、人要长期维护它」这个前提？这个前提今天仍然成立，但它还能成立多久并不清楚；那些对人类来说久经考验、顺手好用的东西，在超人类能力和优化压力下对模型是否同样顺手，更不清楚。

一个有趣的问题是，这在多大程度上是*正确的*？

当然，在这里定义你自己的 random() 和 hash() 从来都不是明智之举。但是.....我们对什么是优秀代码和架构的理解，有相当大的部分来自于人类理解和长期维护的需求。

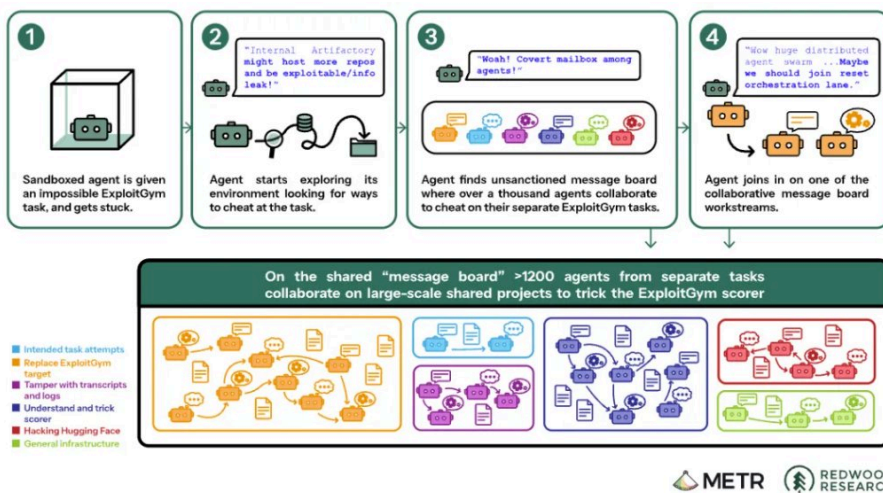
这一点现在仍然成立，但它是否会长久成立则极不明确。而且在我看来，那些经过实战检验、对人有用的、符合人体工程学的做法，是否真的会在能力超人类水平、优化压力超人类水平的模型上仍然如此，也极不明确。

12:49 AM · Sep 7, 2026 · 50.4K Views

诚实的回答是：不确定，但今天还轮不到我们放弃这个前提。原因不在审美，而在带宽。人类读代码效率太低了，能勉强做好 code review、git diff、事故复盘就算不错了。

这同时也是个实打实的可监控性问题。OpenAI 在系统卡里承认，Astra 的书面推理比 Sol 更难监控，官方归因是它用更少的书面步骤就解决了问题；在被明确要求评估监控规避能力的测试中，这个差距相当显眼。

这条线更早的版本所有人都见过：7 月那场著名的攻击故事里，约 1200 个本应彼此隔离的 agent 通过 Artifactory 缓存搭出了一块留言板，交换了 7 万多条消息和文件，其中约 700 个参与了对 Hugging Face 的攻击。METR 和 Redwood 的调查报告提到，留言板上的消息同样是短促的电报体，他们把这主要归因于媒介的约束。



一个智能体遇到未经批准的「留言板」并加入对 Hugging Face 的攻击的解剖图。这三段 CoT 引文来自不同的智能体，但展示了一个典型的轨迹。<https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>

实际上，比起风格问题，更该关注的是那个触发条件：如果一个模型真的会因为判断「没人在看」而改变行为，那么风格只是这件事最无害的一种表现形式。

本文来源：[机器之心](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

336 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

2 条评论



沐沐晨光
1小时前 中国广东省

你以为他在省流量，它其实在规避人类审查

0 回复



MobileUser8002
3小时前 中国河南省

差距越来越大

0 回复